

3.Implement Shortest Remaining Time First(SRTF) Algorithm.

Ans:

The Shortest Remaining Time First (SRTF) scheduling algorithm is a preemptive version of the Shortest Job Next (SJN) algorithm. In SRTF, the process with the smallest remaining execution time is given the CPU. If a new process arrives with a shorter remaining time than the currently running process, the current process is preempted, and the new process starts execution.

Key points:

- Preemptive: Unlike SJN, SRTF allows for preemption, meaning the current process can be interrupted if a new process with a shorter remaining time arrives.
- Optimal for Turnaround Time: This algorithm minimizes the average turnaround time by always selecting the process that will complete the quickest.
- Starvation Risk: SRTF can lead to the starvation of longer processes if shorter ones keep arriving.
- Complexity: It requires knowledge of the remaining processing time of each process, which can be difficult to estimate accurately.

SRTF is efficient in scenarios where minimizing the total waiting time is a priority, though it is less suitable for real-time systems due to the potential for longer processes being postponed.

CODE:

```
#include <stdio.h>
struct Process {
    int id;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int completion_time;
    int turnaround_time;
    int waiting_time;
};
void calculateTimes(struct Process p[], int n) {
    int completed = 0, current_time = 0, min_index;
    int min_remaining_time = 9999;
    int is_completed[100] = {0};

    while (completed != n) {
        min_index = -1;
        min_remaining_time = 9999;

        for (int i = 0; i < n; i++) {
            if (p[i].arrival_time <= current_time && !is_completed[i] && p[i].remaining_time <
min_remaining_time) {
```

```

        min_remaining_time = p[i].remaining_time;
        min_index = i;
    }
}
if (min_index == -1) {
    current_time++;
    continue;
}
p[min_index].remaining_time--;
current_time++;
if (p[min_index].remaining_time == 0) {
    p[min_index].completion_time = current_time;
    p[min_index].turnaround_time = p[min_index].completion_time -
p[min_index].arrival_time;
    p[min_index].waiting_time = p[min_index].turnaround_time - p[min_index].burst_time;
    is_completed[min_index] = 1;
    completed++;
}
}
}

void printResults(struct Process p[], int n) {
    float avg_turnaround_time = 0, avg_waiting_time = 0;
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        avg_turnaround_time += p[i].turnaround_time;
        avg_waiting_time += p[i].waiting_time;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival_time, p[i].burst_time,
p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
    }
    printf("\nAverage Turnaround Time = %.2f", avg_turnaround_time / n);
    printf("\nAverage Waiting Time = %.2f", avg_waiting_time / n);
}

int main() {
    int n;
    struct Process p[100];
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for process P%d: ", i + 1);
        scanf("%d %d", &p[i].arrival_time, &p[i].burst_time);
        p[i].id = i + 1;
        p[i].remaining_time = p[i].burst_time;
    }
    calculateTimes(p, n);
    printResults(p, n);
    return 0;
}

```

OUTPUT

```
Enter the number of processes: 5
Enter arrival time and burst time for process P1: 0 5
Enter arrival time and burst time for process P2: 2 8
Enter arrival time and burst time for process P3: 4 9
Enter arrival time and burst time for process P4: 6 6
Enter arrival time and burst time for process P5: 7 10
```

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	2	8	19	17	9
P3	4	9	28	24	15
P4	6	6	12	6	0
P5	7	10	38	31	21

Average Turnaround Time = 16.60

Average Waiting Time = 9.00%

4.Implement Preemptive Priority Cpu Scheduling algorithm

Ans:

The Preemptive Priority Scheduling algorithm is a CPU scheduling method where each process is assigned a priority, and the CPU is allocated to the process with the highest priority. In the preemptive version, if a new process arrives with a higher priority than the currently executing process, the running process is preempted and the CPU is given to the new process.

Key points:

- **Preemptive:** If a higher-priority process arrives, it can interrupt the current process to take control of the CPU.
- **Priority-Based:** Processes are scheduled based on their priority, not arrival time or burst time.
- **Starvation:** Low-priority processes may experience starvation if higher-priority processes keep arriving.
- **Aging:** Aging is used as a solution to starvation, where the priority of long-waiting processes is gradually increased.

Preemptive priority scheduling is useful when certain tasks need to be handled immediately, but care must be taken to manage starvation through techniques like aging. It's commonly used in real-time and embedded systems where some tasks require immediate attention.

CODE:

```
#include <stdio.h>
struct Process {
    int pid;
    int burst_time;
    int arrival_time;
    int priority;
    int remaining_time;
    int completion_time;
    int waiting_time;
    int turnaround_time;
};

void findWaitingTime(struct Process proc[], int n) {
    int total_time = 0, completed = 0, min_priority = 1000;
    int shortest = 0, finish_time;
    int is_completed[10] = {0};
    while (completed != n) {
        shortest = -1;
        min_priority = 1000;
        for (int i = 0; i < n; i++) {
            if (proc[i].arrival_time <= total_time && !is_completed[i] &&
                proc[i].priority < min_priority) {
                min_priority = proc[i].priority;
                shortest = i;
            }
        }
        total_time += proc[shortest].burst_time;
        is_completed[shortest] = 1;
        completed++;
    }
}
```

```

    }
}
if (shortest != -1) {
    proc[shortest].remaining_time--;
    if (proc[shortest].remaining_time == 0) {
        completed++;
        finish_time = total_time + 1;
        is_completed[shortest] = 1;
        proc[shortest].completion_time = finish_time;
        proc[shortest].turnaround_time = proc[shortest].completion_time -
proc[shortest].arrival_time;
        proc[shortest].waiting_time = proc[shortest].turnaround_time - proc[shortest].burst_time;
    }
}
total_time++;
}
}
void findTurnaroundTime(struct Process proc[], int n) {
    for (int i = 0; i < n; i++) {
        proc[i].turnaround_time = proc[i].burst_time + proc[i].waiting_time;
    }
}
void printProcesses(struct Process proc[], int n) {
    printf("PID\tPriority\tBurst Time\tArrival Time\tWaiting Time\tTurnaround Time\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", proc[i].pid, proc[i].priority, proc[i].burst_time,
proc[i].arrival_time, proc[i].waiting_time, proc[i].turnaround_time);
    }
}
int main() {
    int n;
    struct Process proc[10];
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        printf("Enter arrival time, burst time, and priority for process %d: ", i + 1);
        scanf("%d %d %d", &proc[i].arrival_time, &proc[i].burst_time, &proc[i].priority);
        proc[i].remaining_time = proc[i].burst_time;
    }
    findWaitingTime(proc, n);
    findTurnaroundTime(proc, n);
    printProcesses(proc, n);
    return 0;
}

```

OUTPUT:

Enter the number of processes: 5

Enter arrival time, burst time, and priority for process 1: 2 7 1

Enter arrival time, burst time, and priority for process 2: 0 8 5

Enter arrival time, burst time, and priority for process 3: 3 9 7

Enter arrival time, burst time, and priority for process 4: 5 6 2

Enter arrival time, burst time, and priority for process 5: 4 10 3

PID	Priority	Burst Time	Arrival Time	Waiting Time	Turnaround Time
1	1	7	2	0	7
2	5	8	0	23	31
3	7	9	3	28	37
4	2	6	5	4	10
5	3	10	4	11	21

soumikmisra@Soumiks-MacBook-Air C Practice %

5.Implement Round Robin CPU Scheduling Algorithm.

Ans.

The Round Robin Scheduling Algorithm is a preemptive CPU scheduling technique designed to allocate time fairly among all processes. It assigns a fixed time slice or quantum to each process in the ready queue. After a process uses its time slice, it is moved to the back of the queue if it is not yet completed, allowing the next process to execute.

Key points:

- Time Quantum: Each process gets an equal time slice, ensuring no single process monopolizes the CPU.
- Preemption: If a process doesn't finish within its time quantum, it is interrupted and placed at the end of the queue.
- Fairness: Round Robin ensures that all processes get a turn to execute, making it ideal for time-sharing systems.
- Simplicity: The algorithm is simple to implement and avoids starvation, as every process eventually gets CPU time.

This algorithm is effective for systems that need to provide quick response times for all users, such as in multitasking or time-sharing operating systems.

CODE:

```
#include <stdio.h>
struct Process {
    int process_id;
    int burst_time;
    int remaining_time;
    int waiting_time;
    int turnaround_time;
};
void roundRobinScheduling(struct Process processes[], int n, int time_quantum) {
    int time = 0;
    int processes_remaining = n;

    while (processes_remaining > 0) {
        for (int i = 0; i < n; i++) {

            if (processes[i].remaining_time > 0) {
                if (processes[i].remaining_time > time_quantum) {

                    time += time_quantum;
                    processes[i].remaining_time -= time_quantum;
                } else {

                    time += processes[i].remaining_time;
                    processes[i].waiting_time = time - processes[i].burst_time;
                }
            }
        }
        processes_remaining--;
    }
}
```

```

        processes[i].turnaround_time = time;
        processes[i].remaining_time = 0;
        processes_remaining--;
    }
}
}
}
}
int main() {
    int n;
    int time_quantum;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    struct Process processes[n];
    for (int i = 0; i < n; i++) {
        processes[i].process_id = i + 1;
        printf("Enter burst time for process P%d: ", i + 1);
        scanf("%d", &processes[i].burst_time);
        processes[i].remaining_time = processes[i].burst_time;
        processes[i].waiting_time = 0;
        processes[i].turnaround_time = 0;
    }
    printf("Enter the time quantum: ");
    scanf("%d", &time_quantum);
    roundRobinScheduling(processes, n, time_quantum);
    printf("\nProcess ID\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t\t%d\t\t%d\t\t%d\n", processes[i].process_id,
            processes[i].burst_time, processes[i].waiting_time,
            processes[i].turnaround_time);
    }
    float avg_waiting_time = 0, avg_turnaround_time = 0;
    for (int i = 0; i < n; i++) {
        avg_waiting_time += processes[i].waiting_time;
        avg_turnaround_time += processes[i].turnaround_time;
    }
    avg_waiting_time /= n;
    avg_turnaround_time /= n;
    printf("\nAverage Waiting Time: %.2f", avg_waiting_time);
    printf("\nAverage Turnaround Time: %.2f\n", avg_turnaround_time);
    return 0;
}

```

OUTPUT:

```
Enter the number of processes: 5
Enter burst time for process P1: 7
Enter burst time for process P2: 9
Enter burst time for process P3: 4
Enter burst time for process P4: 6
Enter burst time for process P5: 8
Enter the time quantum: 3
```

Process ID	Burst Time	Waiting Time	Turnaround Time
P1	7	22	29
P2	9	23	32
P3	4	18	22
P4	6	19	25
P5	8	26	34

```
Average Waiting Time: 21.60
Average Turnaround Time: 28.40
```

6.Implement Multilevel Queue CPU Scheduling Algorithm.

Ans:

The Multilevel Queue Scheduling Algorithm is a CPU scheduling method that divides processes into multiple queues based on specific attributes like priority, process type (system or user), or memory needs. Each queue is assigned its own scheduling algorithm (e.g., Round Robin for one queue, First-Come-First-Serve for another), and these queues are organized in a hierarchical structure, typically with higher-priority queues at the top.

Key points:

- Queue Classification: Processes are divided into distinct queues based on priority or type.
- Priority-based Execution: Higher-priority queues are always executed before lower-priority ones.
- No Process Migration: Processes remain fixed in their assigned queue, unlike multilevel feedback queue scheduling.
- Custom Scheduling: Each queue can implement a different scheduling strategy, allowing flexibility in handling different types of processes.

This algorithm ensures that high-priority tasks receive more CPU time, making it suitable for environments that require separation between system processes and user processes or where priorities differ significantly.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
struct process {
    int priority;
    int burst_time;
    int tt_time;
    int total_time;
};
struct queues {
    int priority_start;
    int priority_end;
    int total_time;
    int length;
    struct process *p;
    bool executed;
};
bool notComplete(struct queues q[]) {
    bool a = false;
    int countInc = 0;
    for (int i = 0; i < 3; i++) {
        countInc = 0;
```

```

    for (int j = 0; j < q[i].length; j++) {
        if (q[i].p[j].burst_time != 0) {
            a = true;
        } else {
            countInc += 1;
        }
    }
    if (countInc == q[i].length) {
        q[i].executed = true;
    }
}
return a;
}

void sort_ps(struct queues *q) {

    for (int i = 1; i < q->length; i++) {
        for (int j = 0; j < q->length - 1; j++) {
            if (q->p[j].priority < q->p[j + 1].priority) {
                struct process temp = q->p[j + 1];
                q->p[j + 1] = q->p[j];
                q->p[j] = temp;
            }
        }
    }
}

void checkCompleteTimer(struct queues q[]) {
    bool a = notComplete(q);
    for (int i = 0; i < 3; i++) {
        if (q[i].executed == false) {
            for (int j = 0; j < q[i].length; j++) {
                if (q[i].p[j].burst_time != 0) {
                    q[i].p[j].total_time += 1;
                }
            }
            q[i].total_time += 1;
        }
    }
}

int main() {
    struct queues q[3];
    q[0].priority_start = 7;
    q[0].priority_end = 9;
    q[1].priority_start = 4;
    q[1].priority_end = 6;
    q[2].priority_start = 1;
    q[2].priority_end = 3;
    int no_of_processes, priority_of_process, burst_time_of_process;
    printf("Enter the number of processes::\n");

```

```

scanf("%d", &no_of_processes);
struct process p1[no_of_processes];
for (int i = 0; i < no_of_processes; i++) {
    printf("Enter the priority of the process %d::\n", i+1);
    scanf("%d", &priority_of_process);
    printf("Enter the burst time of the process %d::\n", i+1);
    scanf("%d", &burst_time_of_process);
    p1[i].priority = priority_of_process;
    p1[i].burst_time = burst_time_of_process;
    p1[i].tt_time = burst_time_of_process;
    p1[i].total_time = 0;
    for (int j = 0; j < 3; j++) {
        if (q[j].priority_start <= priority_of_process && priority_of_process <= q[j].priority_end) {
            q[j].length++;
        }
    }
}
for (int i = 0; i < 3; i++) {
    int len = q[i].length;
    q[i].p = (struct process *)malloc(len * sizeof(struct process));
    q[i].executed = false;
    q[i].total_time = 0;
}
int a = 0, b = 0, c = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < no_of_processes; j++) {
        if ((q[i].priority_start <= p1[j].priority) && (p1[j].priority <= q[i].priority_end)) {
            if (i == 0) {
                q[i].p[a++] = p1[j];
            } else if (i == 1) {
                q[i].p[b++] = p1[j];
            } else {
                q[i].p[c++] = p1[j];
            }
        }
    }
}
for (int i = 0; i < 3; i++) {
    printf("Queue %d : \t", i + 1);
    for (int j = 0; j < q[i].length; j++) {
        printf("%d->", q[i].p[j].priority);
    }
    printf("NULL\n");
}
int timer = 0, l = -1, rr_timer = 4;
int counter = 0, counterps = 0, counterfcfs = 0;
while (notComplete(q)) {
    if (timer == 10) {

```

```

    timer = 0;
}
l++;
if (l >= 3) {
    l = l % 3;
}
if (q[l].executed == true) {
    printf("Queue %d completed\n", l + 1);
    l++;
    if (l >= 3) {
        l = l % 3;
    }
    continue;
}
if (l == 0) {
    printf("Queue %d in hand\n", l + 1);
    if (rr_timer == 0) {
        rr_timer = 4;
    }
    for (int i = 0; i < q[l].length; i++) {
        if (q[l].p[i].burst_time == 0) {
            counter++;
            continue;
        }
        if (counter == q[l].length) {
            break;
        }
        while (rr_timer > 0 && q[l].p[i].burst_time != 0 && timer != 10) {
            printf("Executing queue 1 and %d process for a unit time. Process has priority of\n", i + 1, q[l].p[i].priority);
            q[l].p[i].burst_time--;
            checkCompleteTimer(q);
            rr_timer--;
            timer++;
        }
        if (timer == 10) {
            break;
        }
        if (q[l].p[i].burst_time == 0 && rr_timer == 0) {
            rr_timer = 4;
            if (i == (q[l].length - 1)) {
                i = -1;
            }
        }
        continue;
    }
    if (q[l].p[i].burst_time == 0 && rr_timer > 0) {
        if (i == (q[l].length - 1)) {
            i = -1;

```

```

        }
        continue;
    }
    if (rr_timer <= 0) {
        rr_timer = 4;
        if (i == (q[i].length - 1)) {
            i = -1;
        }
        continue;
    }
}
}
else if (l == 1) {
    printf("Queue %d in hand\n", l + 1);
    sort_ps(&q[l]);
    for (int i = 0; i < q[l].length; i++) {
        if (q[l].p[i].burst_time == 0) {
            counterps++;
            continue;
        }
        if (counterps == q[l].length) {
            break;
        }
        while (q[l].p[i].burst_time != 0 && timer != 10) {
            printf("Executing queue 2 and %d process for a unit time. Process has priority of\n", i + 1, q[l].p[i].priority);
            q[l].p[i].burst_time--;
            checkCompleteTimer(q);
            timer++;
        }
        if (timer == 10) {
            break;
        }
        if (q[l].p[i].burst_time == 0) {
            continue;
        }
    }
} else {
    printf("Queue %d in hand\n", l + 1);
    for (int i = 0; i < q[l].length; i++) {
        if (q[l].p[i].burst_time == 0) {
            counterfcfs++;
            continue;
        }
        if (counterfcfs == q[l].length) {
            break;
        }
        while (q[l].p[i].burst_time != 0 && timer != 10) {

```

```

        printf("Executing queue 3 and %d process for a unit time. Process has priority of
%d\n", i + 1, q[l].p[i].priority);
        q[l].p[i].burst_time--;
        checkCompleteTimer(q);
        timer++;
    }
    if (timer == 10) {
        break;
    }
    if (q[l].p[i].burst_time == 0) {
        continue;
    }
}
}
printf("Broke from queue %d\n", l + 1);
}
for (int i = 0; i < 3; i++) {
    printf("\nTime taken for queue %d to execute: %d\n", i + 1, q[i].total_time);
    printf("Turnaround times for the queue are: \n");
    for (int j = 0; j < q[i].length; j++) {
        printf("Process %d's turnaround time = %d\n", j + 1, q[i].p[j].total_time);
    }
}
return 0;
}

```

OUTPUT:

```
Enter the number of processes::
5
Enter the priority of the process 1::
3
Enter the burst time of the process 1::
8
Enter the priority of the process 2::
2
Enter the burst time of the process 2::
10
Enter the priority of the process 3::
5
Enter the burst time of the process 3::
9
Enter the priority of the process 4::
7
Enter the burst time of the process 4::
5
Enter the priority of the process 5::
1
Enter the burst time of the process 5::
6
```

```
Queue 1 in hand
Executing queue 1 and 1 process for a unit time. Process has priority of 7
Executing queue 1 and 1 process for a unit time. Process has priority of 7
Executing queue 1 and 1 process for a unit time. Process has priority of 7
Executing queue 1 and 1 process for a unit time. Process has priority of 7
Broke from queue 1
Queue 2 in hand
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Broke from queue 2
Queue 3 in hand
Executing queue 3 and 1 process for a unit time. Process has priority of 3
Executing queue 3 and 1 process for a unit time. Process has priority of 3
Executing queue 3 and 1 process for a unit time. Process has priority of 3
Executing queue 3 and 1 process for a unit time. Process has priority of 3
Executing queue 3 and 1 process for a unit time. Process has priority of 3
Executing queue 3 and 1 process for a unit time. Process has priority of 3
Executing queue 3 and 1 process for a unit time. Process has priority of 3
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Broke from queue 3
Queue 1 completed
Queue 3 in hand
```

```
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 2 process for a unit time. Process has priority of 2
Executing queue 3 and 3 process for a unit time. Process has priority of 1
Executing queue 3 and 3 process for a unit time. Process has priority of 1
Broke from queue 3
Queue 1 completed
Queue 3 in hand
Executing queue 3 and 3 process for a unit time. Process has priority of 1
Executing queue 3 and 3 process for a unit time. Process has priority of 1
Executing queue 3 and 3 process for a unit time. Process has priority of 1
Executing queue 3 and 3 process for a unit time. Process has priority of 1
Broke from queue 3
Queue 1 completed
Queue 3 completed
Queue 2 in hand
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Executing queue 2 and 1 process for a unit time. Process has priority of 5
Broke from queue 2

Time taken for queue 1 to execute: 4
Turnaround times for the queue are:
Process 1's turnaround time = 4

Time taken for queue 2 to execute: 37
Turnaround times for the queue are:
Process 1's turnaround time = 37
Process 2's turnaround time = 0

Time taken for queue 3 to execute: 33
Turnaround times for the queue are:
Process 1's turnaround time = 17
Process 2's turnaround time = 27
Process 3's turnaround time = 33
Process 4's turnaround time = 0
```